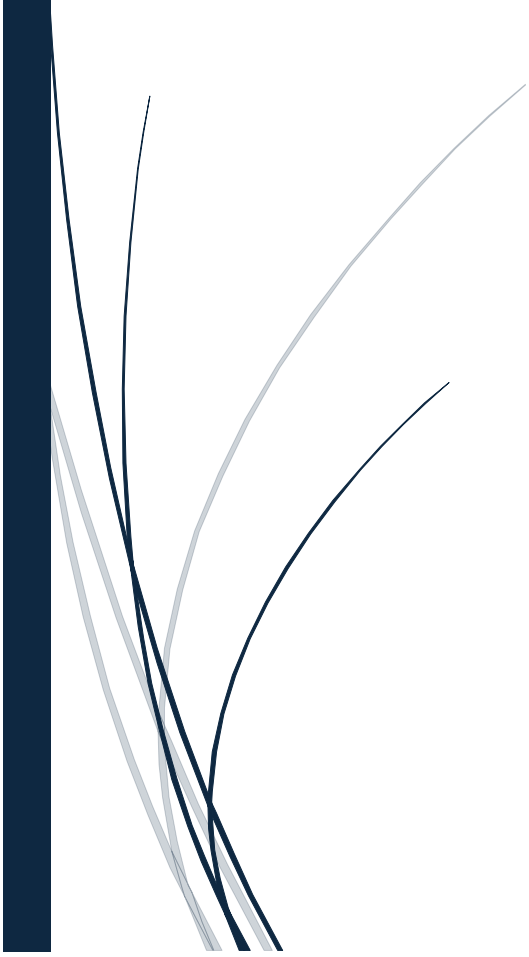
Databases for large datasets

Deepin Pun Magar



1. Introduction

This report evaluates three different databases technologies. These are 3 different genres of databases which are Relational databases, Mongo DB as a document-based database, and Vector database which store data as vector embeddings which are modern databases used in Machine learnings and AI. The aim is to analyses of theses databases and identify their pros and cons for real world applications.

Each database is examined through a case-study involving large scale datasets. This report also highlights which databases are suitable for which types of applications.

2. Case study

2.1 Banking and Transaction Management System (Relational database)

A digital banking system is required to manage customer accounts, financial transactions, and related transactions. The system must handle a high volume of daily transactions such as deposits, withdrawals and fund transfers.

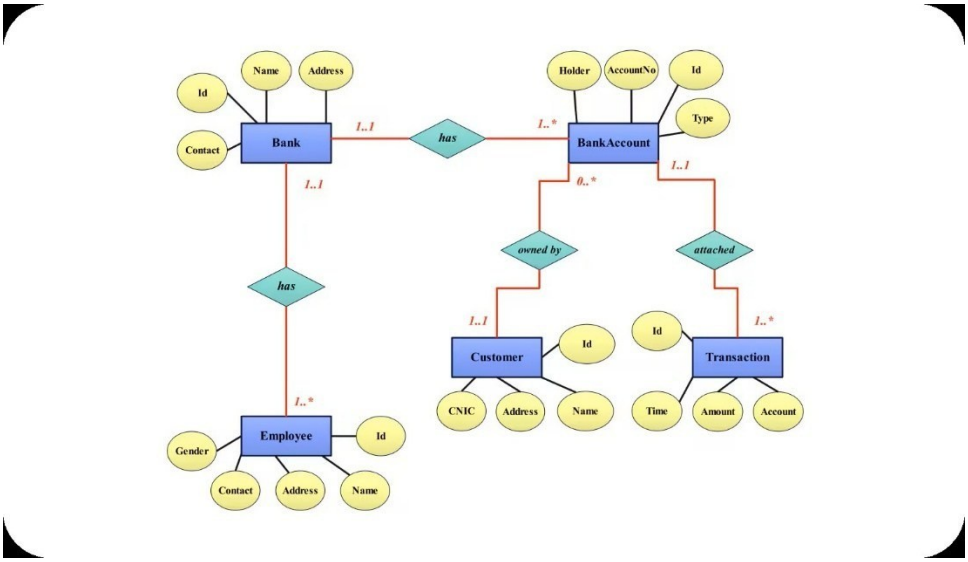


Image 1: ER diagram of bank management system

We are having:

- One to many cardinality
- Many to one cardinality
- Many to many cardinality

The data in this system is highly structured and relational. For example, each customer has multiple transactions. Their relationship must be consistently maintained and enforced. Each table has primary key for data integrity. Every table must be linked to each other for different operations. Data integrity and Data consistency are the main concerns for banking system.

In a banking environment, a “commit” must be absolute. However, writing directly to the main database file (data pages) on every transaction is incredibly slow. RDBMS solves this using Write-Ahead logging**.** Before any changes are made to the actual database records, the database describes the intended change in sequential log file (the WAL). The system guarantees that the log entry is flushed to stable storage disk before the transaction is conformed to the user. If the power cuts mid-transactions, the RDBS performs a “redo” or “undo” operation upon reboot by reading the WAL. It replays the successful transactions that didn’t make it to the data files and rolls back incomplete ones.

Banks need high throughput—thousands of people checking balances while thousands more are making purchases. Traditional "locking" (where a reader blocks a writer) would cause the system to grind to a halt. Most modern RDBMS (like PostgreSQL or Oracle) use MVCC. When a transaction starts, a “snapshot” of the database is given at that specific point in time (a specific System Change Number or Transaction ID). Readers do not block writers, and writers do not block readers. If you are reading your balance while a deposit is being processed, you see the "old" version until the "new" version is fully committed. Banks typically operate at the Serializable isolation level, the highest level. This mathematically ensures that the result of several concurrent transactions is the same as if they had been executed one after another, eliminating "phantom reads" or "write skew."

For a bank, searching for a transaction among billions must be fast and, more importantly, deterministic. RDBMS uses B-Trees (Balanced Trees) for indexing. Unlike hash-based systems that can have collisions, a B-Tree ensures that the time taken to find a record (logarithmic time complexity, or $O (\log n) $) remains stable as the dataset grows, ensuring predictable performance during peak banking hours.

NoSQL databases, such as document-based or key-value systems, are less suitable for banking applications due to their weaker consistency models. While they offer scalability and flexibility, they may not guarantee the same level of transactional reliability as relational databases. In financial systems, where precision and consistency are essential, this limitation makes NoSQL databases a less appropriate choice.

2.2 E-commerce Databases (MongoDB)

It is a document-oriented database which stores data in BSON (binary JSON) documents. Different products have vastly different sets of attributes, so it is hard to store them in the same table, building a table structure to store such data becomes difficult. It offers flexibility and rapid scaling. E-commerce sites need to store unstructured data which does not fit perfectly in the table and rows. For example: what user clicks, hovered over, and their cart, it provides flexibility to store them in single document.

The first is to design a schema for website.

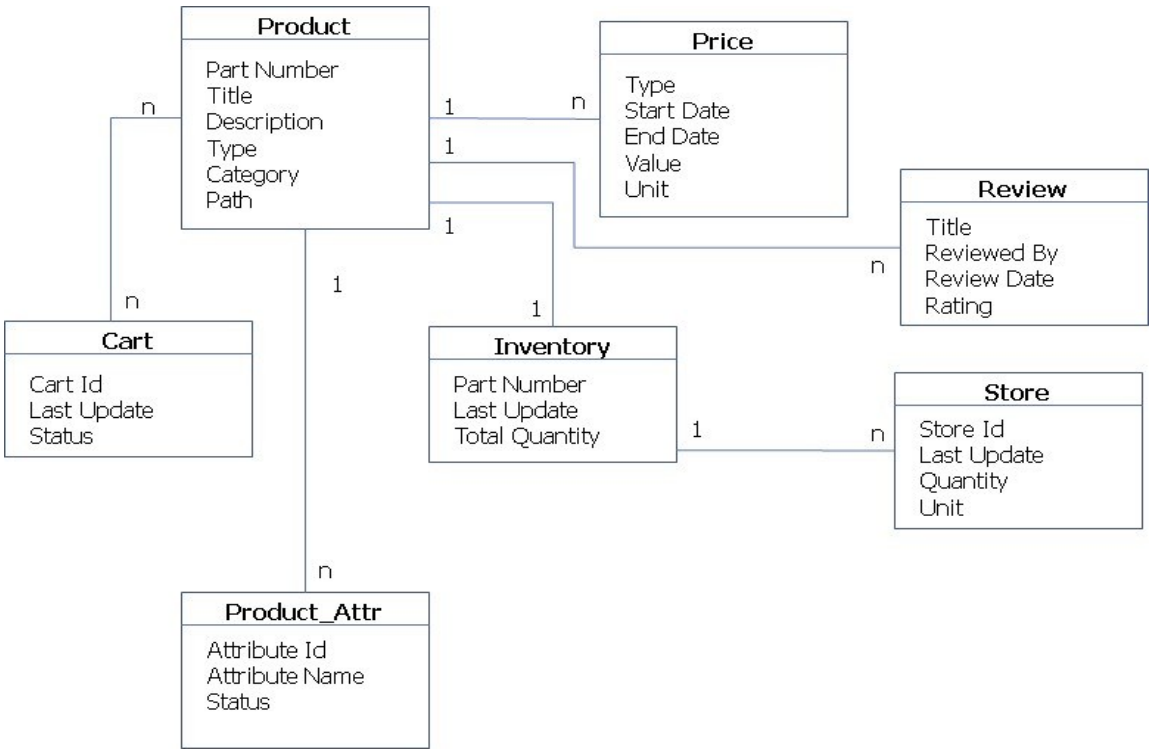


Fig: Schema of Ecommerce site

This is a relational schema (ER diagram). I am going to explain how MongoDB (document based) handle this differently. It does not split data into many tables, instead it combines related data into single document in JSON line format.

Here is the MONGODB version of my Schema:


```
{
  "product_id": "P101",
  "title": "iPhone 15",
```

```
"description": "Latest Apple smartphone",
"category": "Electronics",
"price": [  {
    "type": "discount",
    "value": 999,
    "unit": "USD",
    "start_date": "2026-01-01",
    "end_date": "2026-02-01"
  }  ],

"attributes": {
    "attribute_name": "RAM",
    "value": "8GB   },
  {
    "attribute_name": "Storage",
    "value": "128GB"
  }  ],

"reviews": [   {
    "title": "Great phone",
    "reviewed_by": "User123",
    "rating": 5,
    "date": "2026-03-10"
  }  ],
"inventory": {
  "total_quantity": 500,
  "stores": [   {
```

```
  "store_id": "S1",
  "quantity": 200
},  {
  "store_id": "S2",
  "quantity": 300
}  ]  }  }
```


The first main difference is that it does not store data in different tables. It stores related data in one page in JSON like format. Due to this format, there is advantage of flexible schema which is effective for ecommerce sites because each product has different fields. In relational databases, there is a fixed column, which does not purely justify the data.

MongoDB allows flexible schemas, enabling different products to have different attributes without requiring schema modifications. It approaches document-oriented approach, which allows us to store related data in a single document, which reduces the need for JOIN operation and improves query performance.

E-commerce traffic is notoriously "spiky". It is built to scale via Sharding. It automatically distributes data across a cluster of community servers. During high traffic, you add more nodes to the cluster to share a load.

Therefore, for a dynamic and rapidly evolving system such as an e-commerce platform, MongoDB provides a more practical and scalable solution.

2.3 Vector database for semantic search

Vector databases are designed for meaningful matches. This is the core of semantic search, which understands the intent and context of the query rather than just matching keywords. Vector databases enable similarity-based search using mathematical distance measurements such as cosine similarity or Euclidean distance. They are widely used in artificial intelligence and machine learning applications where data is represented as embeddings.

A vector database is the most effective in this scenario due to its design to handle highdimensional vector data and perform similarity-based queries efficiently. To perform a semantic search image, text etc. should be converted to Vector Embeddings. An embedding is the long list of numbers, which represent the raw data in vector space. In semantic search context of the prompt should be relevant. The co-ordinations of contextually relevant results will be very close in the Vector space. Unlike keyword search, matching words or sentences does not make relevance in the semantic search. It is done by mathematics computation. A vector database is the container of those mathematics computations. It is the latest and most advanced form of database for machine learning and AI. It generally works as a memory for Artificial Intelligence.

Mathematically, a vector {v} represents a point in n-dimensional space:

{v} = [x1, x2, x3,……,xn]

The data in this system is unstructured and represented as high-dimensional vectors rather than traditional rows or documents. A vector database indexes those values so they can be searched incredibly fast.

The workflow looks like this:

a) Indexing: The database takes the vector and stores it alongside side with metadata.

b) Querying: When users search for something, the database converts that into vectors using the same mechanism.

c) Similarity search: The database looks for the nearest Neighbors, the vectors that are mathematically closest to your query vector.

The distance between two vectors determines the similarity between two objects or context. Many techniques are used to make queries faster such as clustering vector datapoints according to distance, determining the closest vectors using Approximate Nearest Neighbour (ANN).

Relational databases are not effective for this type of application because they are designed for structured data and exact-match queries. To find the nearest neighbor on SQL based database with 10 million rows, the computer has to calculate the distance for every single row one by one. It is too slow to real time app or any commercial app.

Similarly, document-based databases such as MongoDB are designed to store flexible, semistructured data but are not optimized for Vector similarity search. Therefore, for applications that rely on semantic understanding and similarity-based retrieval, vector databases provide a significantly more efficient and scalable solution compared to relational and document-based databases.

3 REFERENCES

Elmasri, R., & Navathe, S. B. (2016). Fundamentals of database systems (7th ed.). Pearson.

Date, C. J. (2003). An introduction to database systems (8th ed.). Pearson.

MongoDB, Inc. (2024). MongoDB documentation. https://www.mongodb.com/docs/

Liu, Y., Chen, X., & Wang, J. (2023). Vector database management systems: A survey. arXiv. https://arxiv.org/abs/2309.12345

Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). Database system concepts (7th ed.). McGraw-Hill.